

Daily Frontend + Backend Schedule — Narri E-Commerce (30 Days)

Each day lists what to build on **Backend** and **Frontend** so you always finish the day with a connected, working slice — not two disconnected halves.

Buffer days (7, 14, 21, 28) have lighter loads on purpose — use them to fix what slipped, not to start new features.

Week 1 — Foundation, Auth, Product core

Day	Backend	Frontend
1	Init Express server, connect MongoDB Atlas, folder structure. Deploy empty API to Render.	Init React + Vite, Tailwind setup, folder structure. Deploy empty app to Vercel.
2	Define Mongoose schemas: User, Product, Category, Order, Review, Banner.	Set up React Router, build empty page shells: Home, Product, Cart, Checkout, Login/Signup.
3	Auth APIs: signup, login, bcrypt password hashing, JWT generation.	Signup & Login page UI — forms with react-hook-form, client-side validation.
4	Auth middleware: <code>protect</code> (verify JWT) and <code>adminOnly</code> (role check).	Connect signup/login forms to API, store token, build AuthContext + protected route wrapper.
5	Product CRUD APIs (create/list/detail/update/delete) + Category CRUD APIs.	Homepage: fetch and render product grid from API.
6	Add filtering/sorting/pagination query params to Product API.	Product detail page: image gallery, price, description, size selector (static).
7	BUFFER — fix auth bugs, test all Week 1 API endpoints in Postman.	BUFFER — fix UI bugs, check mobile responsiveness of pages built so far.

Week 2 — Storefront: browsing, cart, checkout, payment

Day	Backend	Frontend
8	Add text search index + search endpoint.	Wire category filters, price filter, and sort dropdown to Product API.
9	Review Order schema fields, no new endpoints yet.	Cart state (Zustand/Context) with localStorage persistence; build Cart page UI.
10	Order creation API: <code>POST /orders</code> — calculate itemsPrice/shippingPrice/totalPrice.	Checkout page: address form + order summary UI.
11	Razorpay/Stripe integration: create payment order/intent endpoint.	Connect checkout submit to create-order API, integrate payment widget SDK.
12	Payment webhook handler — verify signature, mark order <code>paid</code> , update status.	Order confirmation page; handle payment success/failure redirect states.
13	Order APIs: <code>GET /orders/mine</code> , <code>GET /orders/:id</code> .	"My Orders" page — list view + single order detail view.
14	BUFFER — test full payment flow with test API keys, fix webhook edge cases.	BUFFER — test full purchase journey (browse → cart → pay → confirmation), fix bugs.

Week 3 — Admin dashboard: products, banners, reviews

Day	Backend	Frontend
15	Harden admin middleware; write a couple of manual tests with a non-admin token.	Admin dashboard shell — sidebar nav, layout, protected admin-only routes.
16	Add stock-update endpoint if missing; confirm Product API covers admin needs.	Admin Product management UI — list, add, edit, delete, stock field.
17	Banner CRUD API + Cloudinary image upload endpoint.	Admin Banner management UI — upload image, set position/order, toggle active.
18	Add <code>GET /banners?active=true</code> endpoint sorted by order.	Connect homepage hero section to live banner API (carousel).
19	Review API: create review, list by product, approve/reject flag.	Reviews section on product page — display approved reviews + submit-review form.
20	Refine review approval logic (one review per user per product).	Admin Review moderation UI — table with approve/reject/delete actions.
21	BUFFER — fix banner/review API bugs.	BUFFER — test banner + review flows end-to-end, polish admin UI.

Week 4 — Orders/sales dashboard, polish, launch

Day	Backend	Frontend
22	Admin Orders API — list all orders (filter by status), update order status.	Admin Orders page — table, status filter, "update status" action.
23	Sales summary API — aggregate total revenue, order count.	Admin dashboard stat cards — revenue, order count, pending reviews, active banners.
24	Add missing DB indexes, quick performance pass on slow queries.	Mobile responsiveness pass — storefront (home, product, cart, checkout).
25	Standardize API error responses (consistent shape, no stack traces).	Mobile responsiveness pass — admin dashboard; add loading/error states everywhere.
26	Join frontend for full manual QA: signup → browse → cart → checkout → payment → order visible in admin. Fix bugs found.	Same QA pass from the UI side — note every bug, fix as you go.
27	Fix remaining bugs from Day 26.	Cross-browser/device check (Chrome + Safari mobile especially).
28	BUFFER — final backend catch-up.	BUFFER — final frontend catch-up.
29	Set production env vars, deploy final backend build, test one real transaction.	Production build, deploy frontend + admin, verify live site end-to-end.
30	Client walkthrough, handover, buffer for last-minute fixes.	Client walkthrough, handover, buffer for last-minute fixes.

How to use this day to day

- Do backend work first thing each day, then immediately wire it into the frontend task listed for that same day — that's the "vertical slice" from the plan.
- If a day's backend task is light, use the extra time to get ahead on tomorrow's frontend task rather than leaving it idle.
- If you lose a full day, shift everything after it back by one and absorb the loss at the nearest buffer day (7/14/21/28).